Com base nos documentos enviados, preparei uma análise direta sobre a situação atual do projeto **Plataforma de carona compartilhada (FATEC Araçatuba)** para ajudar a equipe (Mariana, Aline, Bruno, João e Ryan) nos próximos [1]

Repositório criado com commits do dia e README inicial (mesmo que raso).

Parabéns pelo primeiro passo! Ter o repositório configurado com os commits do dia e um README.md inicial é a base perfeita para manter o projeto organizado.

Para que o seu repositório fique ainda mais profissional e fácil de retomar nos próximos dias, aqui estão os próximos passos recomendados:

1. Incremente o README aos poucos

Mesmo que esteja raso agora, você pode aplicar uma estrutura simples usando Markdown conforme o projeto evolui. Um bom padrão básico é:

- **Título do Projeto**: O nome e uma linha explicando o que ele faz.
- **Como Executar**: Os comandos necessários para rodar o projeto localmente (ex: npm install ou pip install -r requirements.txt).
- **Tecnologias**: Uma lista simples das ferramentas utilizadas (ex: Python, React, PostgreSQL).

2. Mantenha a frequência de Commits

- **Commits pequenos e frequentes**: Evite acumular muitas alterações para commitar tudo de uma vez. Faça commits a cada funcionalidade terminada ou bug corrigido.
- **Mensagens claras**: Use o padrão do *Conventional Commits* se quiser deixar o histórico impecável (ex: feat: adiciona rota de login, docs: atualiza o readme).

3. Próximo passo sugerido

Se o seu código já está no GitHub/GitLab, o próximo passo ideal é criar um arquivo .gitignore para garantir que arquivos desnecessários (como pastas node_modules, ambientes virtuais .venv ou chaves de API secretas) não sejam enviados por acidente.

Deseja ajuda para estruturar o README ou criar um .gitignore específico para a linguagem que você está usando?

Deseja ajuda para estruturar o README ou criar um .gitignore específico para a linguagem que você está usando?

Deseja ajuda para estruturar o README ou criar um .gitignore específico para a linguagem que você está usando?

Com base no contexto do projeto de vocês (**Plataforma de carona compartilhada da FATEC Araçatuba**), o repositório já utiliza **HTML puro, CSS e JavaScript** implantado via **Jekyll no GitHub Pages**. [1]

Como o README.md original estava com espaços em branco, preparei um modelo completo, profissional e pronto para colar no repositório, além do arquivo .gitignore ideal para essa estrutura.

📄 Modelo de README.md Profissional

Substitua o conteúdo do seu arquivo atual por este texto em Markdown:

markdown

```
# FatecCar - Plataforma de Carona Compartilhada

**Projeto Interdisciplinar | FATEC Araçatuba**

Foco em mobilidade urbana, sustentabilidade e integração da comunidade acadêmica.

## 📌 Sobre o Projeto

O **FatecCar** é um protótipo estático (mockup front-end) voltado para conectar alunos que oferecem e buscam caronas para a faculdade. O objetivo é reduzir a pegada de carbono, otimizar os custos de transporte dos estudantes e mitigar problemas de trânsito locais.

> **Status do Projeto:** 🚀 Protótipo Acadêmico Publicado via GitHub Pages.

## 🛠️ Tecnologias Utilizadas

* **Front-end:** HTML5, CSS3 (Design Responsivo), JavaScript (ES6)

* **Deploy & CI/CD:** GitHub Actions / Jekyll (GitHub Pages)

## 📱 Funcionalidades do Protótipo
```

* **Busca Inteligente:** Fluxo de simulação de pesquisa de caronas por bairros e horários.

* **Resultados e Filtros:** Exibição dinâmica de motoristas disponíveis de forma simulada.

* **Solicitação Direta:** Interface de confirmação com geração de número de protocolo do pedido.

* **Interface Mobile-First:** Totalmente adaptado para navegação em smartphones.

🚀 Como Executar o Projeto Localmente

Como este projeto é um protótipo estático, você não precisa de nenhum compilador ou servidor backend.

1. Clone o repositório:

```bash
git clone https://github.com
```

2. Navegue até a pasta do projeto.

3. Abra o arquivo `index.html` diretamente em qualquer navegador web.

👥 Equipe

* Mariana

* Aline

* Bruno

* João

* Ryan

Arquivo .gitignore Recomendado

Como o GitHub Actions do projeto está configurado para usar o construtor do **Jekyll**, é muito comum que pastas temporárias de compilação surjam na máquina

de quem rodar o projeto. Crie um arquivo chamado .gitignore na raiz do repositório com o seguinte conteúdo: [1]

text

```
# Pastas de build do Jekyll
.jekyll-cache/
_site/
.jekyll-metadata

# Arquivos do sistema operacional
.DS_Store
Thumbs.db

# Configurações locais de IDEs (opcional)
.vscode/
.idea/
```